Software Design Specification

Carefree Aiding Caretakers

PRESENTED BY 24K1010,24K0688,24K0812

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 System Overview

Carefree — Aiding Caretakers is a mobile application built for Android devices, designed to streamline and automate the complex, time-intensive tasks faced by in-home caregivers. The application serves two primary user roles: Caretakers (the primary users who manage patient care) and Guardians (emergency contacts or family members who monitor patient well-being).

The frontend is developed using the Flutter framework, which connects the user interface to the backend via REST APIs. The backend is built on Java (Spring Boot) for core business logic, with Python handling all Machine Learning and Artificial Intelligence components via FastAPI. A MySQL database stores all patient, caretaker, and operational data.

1.2 Problem Statement

In-home caregivers manage a wide range of responsibilities — from tracking patient vitals and administering medications, to scheduling appointments, maintaining dietary plans, and communicating with guardians. These tasks are typically handled through fragmented tools such as paper notes, spreadsheets, and informal messaging apps, leading to information loss, miscommunication, and potential medical errors.

Carefree addresses this gap by providing a unified, structured digital platform that centralizes all caretaking activities, supports AI-assisted features such as meal flagging and cost prediction, and enables real-time communication between caretakers and guardians.

1.3 Scope

This Software Design Document (SDD) covers the complete design of the Carefree mobile application. It includes:

- System architecture and service decomposition
- UML diagrams: Use Case, Class, Sequence, and State Chart
- Logical, Interface, Interaction, and State Dynamic viewpoints
- System features and functional/non-functional requirements
- Deployment and API integration details

The document excludes implementation code, test plans, and post-deployment operations.

1.4 Objectives

- Provide a comprehensive, implementable design for all development team members
- Ensure alignment with IEEE Std 1016-2009 standards
- Document all system features, flows, and constraints clearly
- Support the use of ML/AI components (Medical Cost Predictor and Meal Flagger)
- Enable secure, role-based access for Caretakers and Guardians

1.5 Users and Stakeholders

Stakeholder	Role	Concerns
Caretaker	Primary user	Ease of use, data accuracy, task management, notifications
Guardian	Secondary user	Patient visibility, communication, report access
Developers	System builders	Clear design specs, modular architecture, testability
Testers/QA	Validation team	Feature completeness, requirement traceability
Patient	Indirect beneficiary	Accurate care delivery, privacy, data security

1.6 System Requirements Summary

The system must support the following high-level capabilities:

Feature	Description
Patient Information Management	CRUD operations on patient profiles with lab reports and visit logs
Vitals & Symptom Tracking	Daily recording of vitals with trend analysis and alerts
Medication & Appointments	Schedule, edit, delete medications and appointments with reminders
Caretaker Task Management	To-do list system with priority, frequency, and progress tracking
Dietary Plan Management	Customized meal plans with nutritional calculations
Expense Tracker	Log, edit, delete expenses and generate financial reports
Guardian Communication	In-app messaging with document attachment support
Custom Notifications	Configurable reminders via push, SMS, or email
Report Generation	Vitals, symptom, and financial reports exportable as PDF/CSV
ML Medical Cost Predictor	Gradient boosting model for yearly cost prediction
AI Meal Flagger	Image-based meal detection with dietary restriction checking

1.7 Software Process Model

Carefree adopts an Iterative and Incremental development model, aligned with principles from the Unified Process (UP).

Rationale:

- The system encompasses distinct, independently deployable features (e.g., vitals tracking, expense management, AI meal flagging), making iterative delivery natural and practical.

- AI/ML components (cost predictor, meal flagger) require prototyping and feedback cycles that benefit from iterative refinement.
- Guardian and Caretaker roles have different access needs, which can be developed and validated in parallel increments.
- The three-tier Service-Oriented Architecture (SOA) supports modular development — each microservice can be built, tested, and integrated independently.

Key phases: Inception (requirements and use cases), Elaboration (architecture and detailed design), Construction (iterative feature implementation), Transition (deployment and testing).

CHAPTER 2: SYSTEM ANALYSIS AND MODELING

2.1 Use Case Diagram

The Carefree system defines two primary actors: the Caretaker and the Guardian. The Caretaker is the primary user with full access to all features, while the Guardian has read-only or limited access to patient data and reports.

Key use case relationships include:

- <<include>>: Mandatory sub-functions automatically triggered (e.g., Login includes Validate Credentials; Add Appointments includes Create Notification Reminders)
- <<extend>>: Optional sub-functions triggered under specific conditions (e.g., Log Symptoms extends to Add Custom Options if customized; AI Meal Flagger extends to Enter Meal if AI cannot detect)

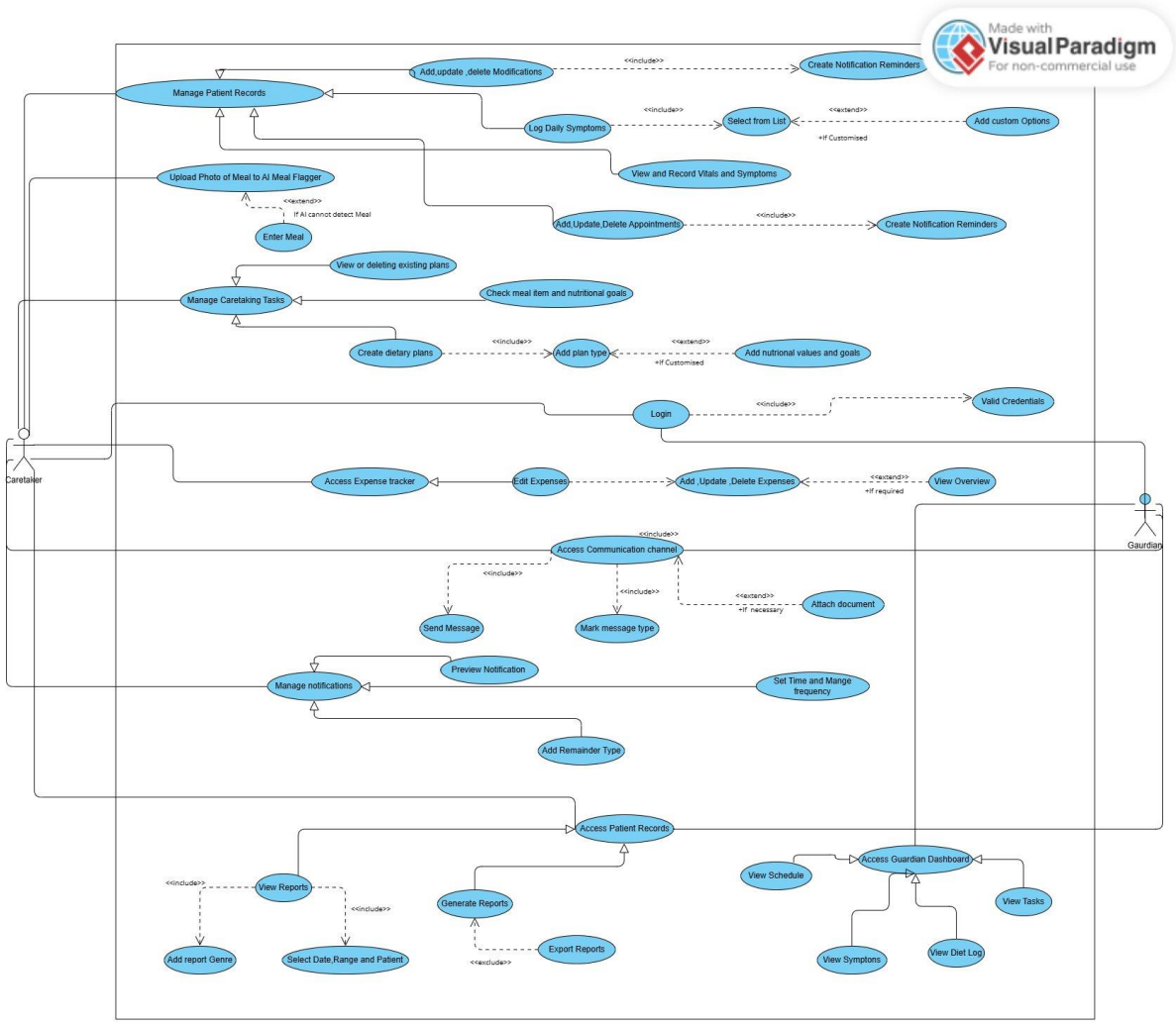


Figure 1. Use Case Diagram — Carefree System

2.1.1 Use Case Description Table

Use Case	Actor(s)	Description	Relationships
Login	Caretaker, Guardian	Authenticate user using credentials stored in the database	<<include>> Validate Credentials
Manage Patient Records	Caretaker	Add, update, delete patient information including vitals, symptoms, medications	<<include>> Add/Update/Delete Modifications
Log Daily Symptoms	Caretaker	Record patient symptoms daily for health trend tracking	<<include>> Select from List; <<extend>> Add Custom Options
Add/Update/Delete Appointments	Caretaker	Manage patient appointment scheduling	<<include>> Create Notification Reminders
View & Record Vitals and Symptoms	Caretaker	Track and review vitals with alert generation	—
Upload Photo to AI Meal Flagger	Caretaker	Use AI to identify meals and check dietary restrictions	<<extend>> Enter Meal (if AI cannot detect)
Manage Caretaking Tasks	Caretaker	Organize daily caregiving duties with priority and frequency	<<include>> Create Dietary Plans, Check Meal Items
Access Expense Tracker	Caretaker	Monitor and manage all patient-related expenses	<<include>> Edit Expenses; <<extend>> View Overview
Access Communication Channel	Caretaker, Guardian	In-app messaging between caretaker and guardian	<<include>> Send Message, Mark Message Type; <<extend>> Attach Document
Manage Notifications	Caretaker, Guardian	Configure and schedule reminders for tasks, appointments, medications	<<include>> Preview Notification, Add Reminder Type, Set Time & Frequency
Access Patient Reports	Caretaker, Guardian	View and generate analytical reports on patient health and finances	<<include>> View Reports, Generate Reports; <<extend>> Export Reports
Access Guardian Dashboard	Guardian	Monitor patient schedule, tasks, symptoms, and diet logs	<<include>> View Schedule, Tasks, Symptoms, Diet Log

2.2 Class Diagram

The class diagram represents the object-oriented data model of Carefree. It shows 16 classes with their attributes, methods, and inter-class relationships including association, aggregation, composition, and inheritance.

The BCE (Boundary-Control-Entity) model is reflected as follows:

- Boundary classes: The Flutter UI components (Login, Dashboard, Vitals Page, etc.) interact with the user
- Control classes: Service layer classes (Patient Management Service, Dietary Management Service, etc.) handle business logic
- Entity classes: Person, Patient, Guardian, Caretaker, Medication, Vitals, Symptoms, Appointment, Task, Notification, DietaryPlan, Meal, Expense, Cardiac, Respiratory, Others

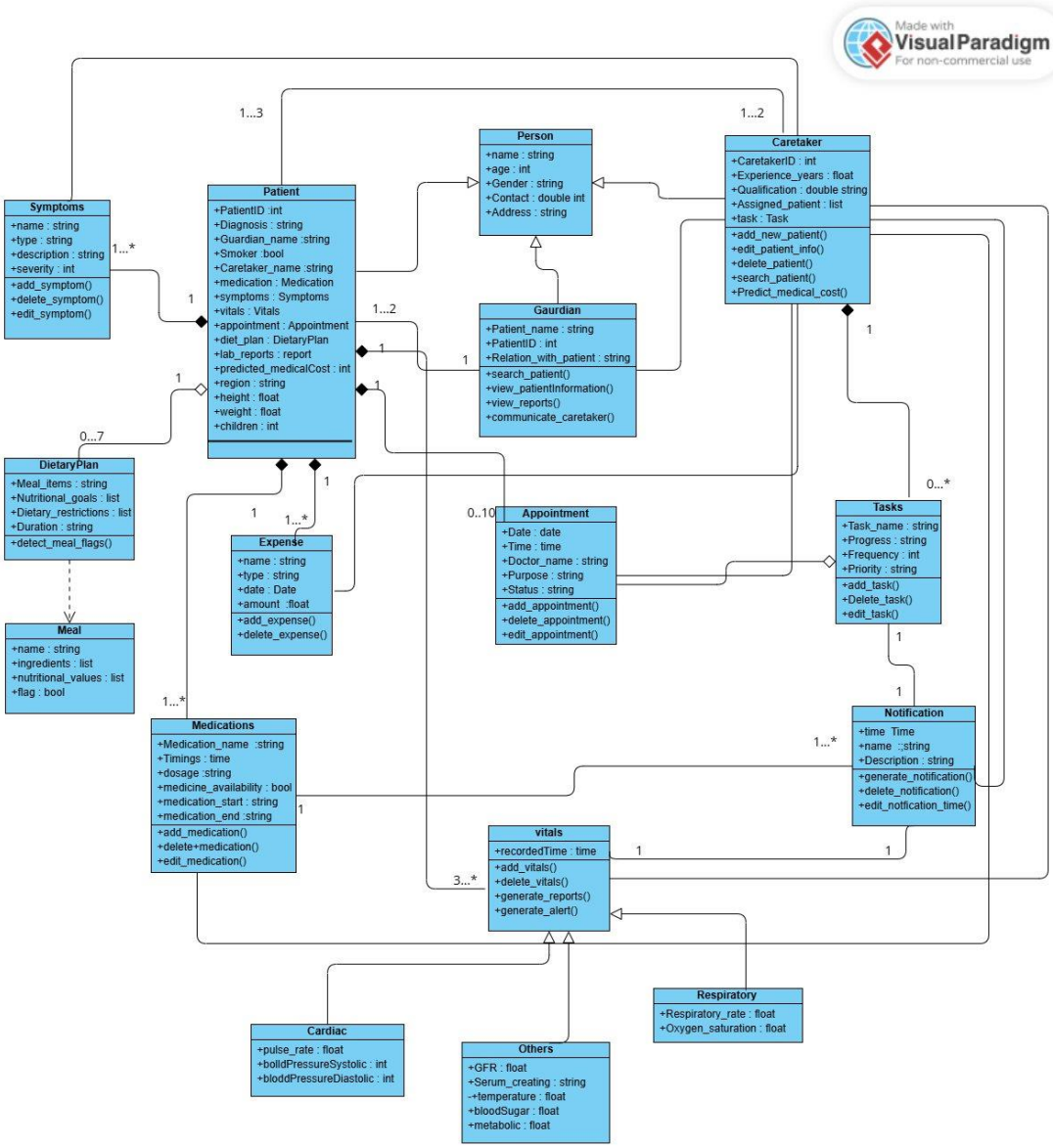


Figure 2. Class Diagram — Carefree System

2.2.1 Class Descriptions

Class	Type	Key Attributes	Key Methods
Person	Entity (Abstract)	name, age, gender, contact, address	—
Patient	Entity	PatientID, diagnosis, guardian_name, medication, vitals, appointment, diet_plan, predicted_medicalCost, smoker, region, height, weight, children	—
Guardian	Entity	Patient_name, PatientID, Relation_with_patient	search_patient(), view_patientInformation(), view_reports(), communicate_caretaker()
Caretaker	Entity	CaretakerID, Experience_years, Qualification, Assigned_patient, task	add_new_patient(), edit_patient_info(), search_patient(), delete_patient(), Predict_medical_cost()
Medication	Entity	Medicine_name, Timings, dosage, medicine_availability, medication_start, medication_end	add_medication(), delete_medication(), edit_medication()
DietaryPlan	Entity	Meal_items, Nutritional_goals, Dietary_restrictions, Duration	detect_meal_flags()
Meal	Entity	name, ingredients, nutritional_values, flag	—
Task	Entity	Task_name, Progress, Frequency, Priority	add_task(), delete_task(), edit_task()
Appointment	Entity	Date, Time, Doctor_name, Purpose, Status	add_appointment(), delete_appointment(), edit_appointment()
Symptoms	Entity	name, type, description, severity	add_symptom(), delete_symptom(), edit_symptom()
Vitals	Entity (Parent)	recordedTime	add_vitals(), delete_vitals(), generate_reports(), generate_alert()
Cardiac	Entity (Child of Vitals)	pulse_rate, bloodPressureSystolic, bloodPressureDiastolic	—
Respiratory	Entity (Child of Vitals)	Respiratory_rate, Oxygen_saturation	—

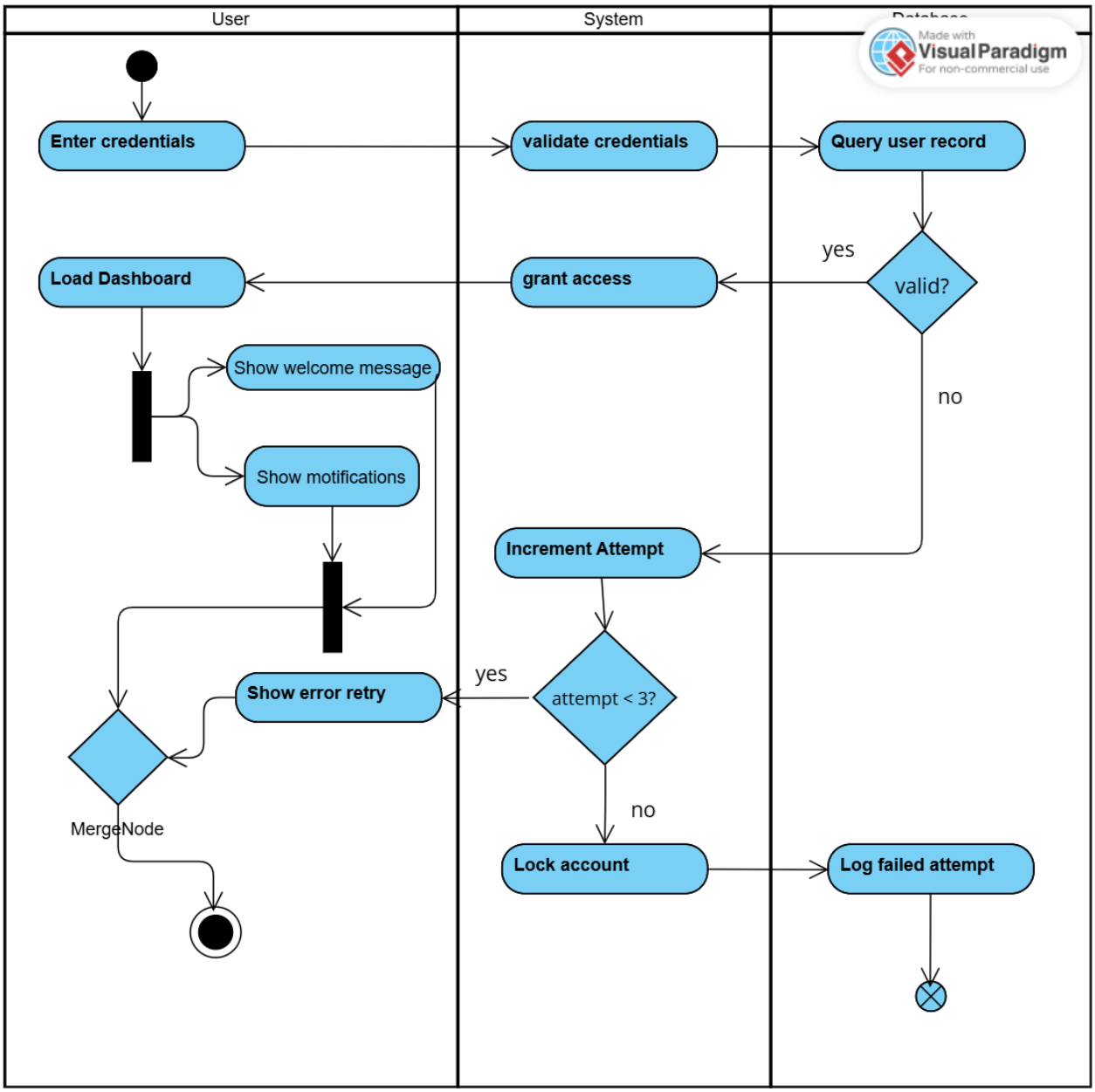
Others	Entity (Child of Vitals)	GFR, Serum_creatinine, temperature, bloodSugar, metabolic	—
Notification	Entity	time, name, description	generate_notification(), delete_notification(), edit_notification_time()
Expense	Entity	name, type, date, amount	add_expense(), delete_expense()

2.2.2 Key Relationships

Relationship	Classes Involved	Multiplicity	Type
Person generalizes Patient, Guardian, Caretaker	Person → Patient, Guardian, Caretaker	1 to 1..2	Inheritance
Patient has Symptoms	Patient → Symptoms	1 to 1..*	Composition
Patient has Medication	Patient → Medication	1 to 1..*	Composition
Patient has Appointment	Patient → Appointment	1 to 0..10	Aggregation
Patient has DietaryPlan	Patient → DietaryPlan	1 to 0..7	Composition
Patient has Expense	Patient → Expense	1 to 1..*	Aggregation
Caretaker manages Tasks	Caretaker → Tasks	1 to 0..*	Association
Task has Notification	Task → Notification	1 to 1..*	Association
Vitals is parent of Cardiac, Respiratory, Others	Vitals → Cardiac, Respiratory, Others	3..* to 1	Inheritance
DietaryPlan references Meal	DietaryPlan → Meal	1 to *	Dependency
Guardian linked to Patient	Guardian → Patient	1 to 1..2	Association

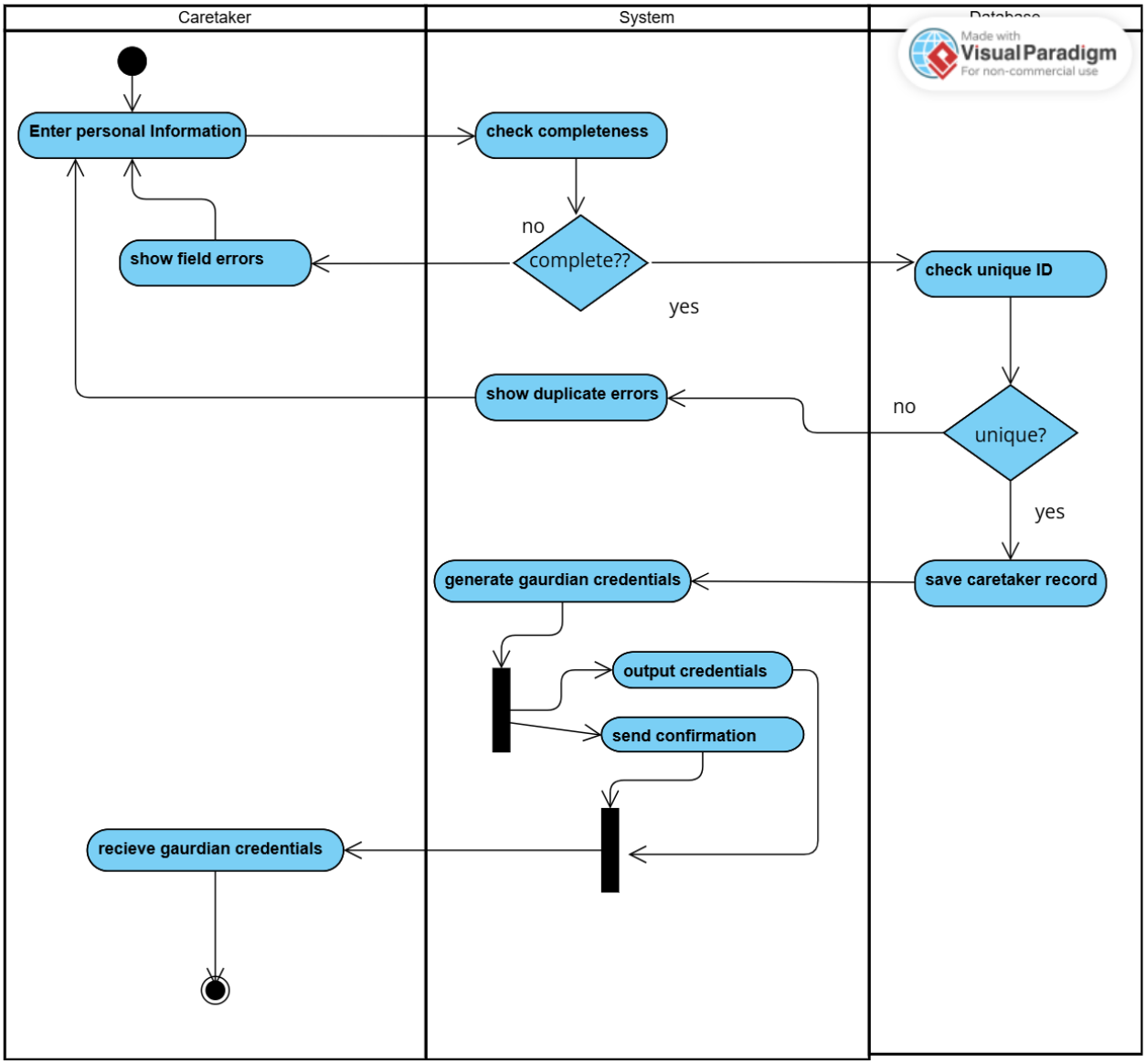
2.3 Activity Diagrams

2.3.1 Login



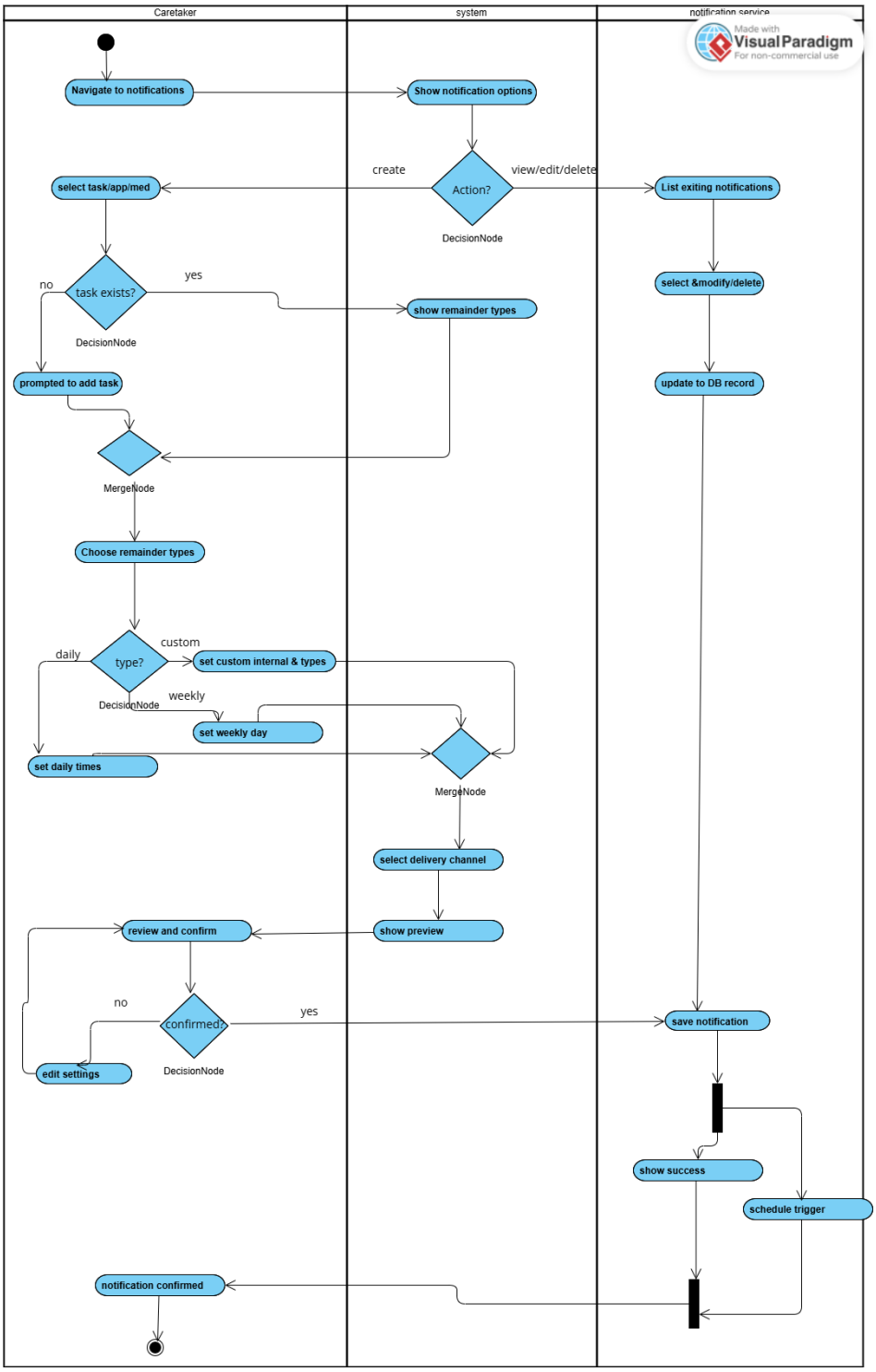
The user opens the application and enters their email and password. The system validates the credentials with the database. If the information is correct, the user is granted access to the dashboard. If invalid, an error message is displayed and the user retries.

2.3.2 Registration



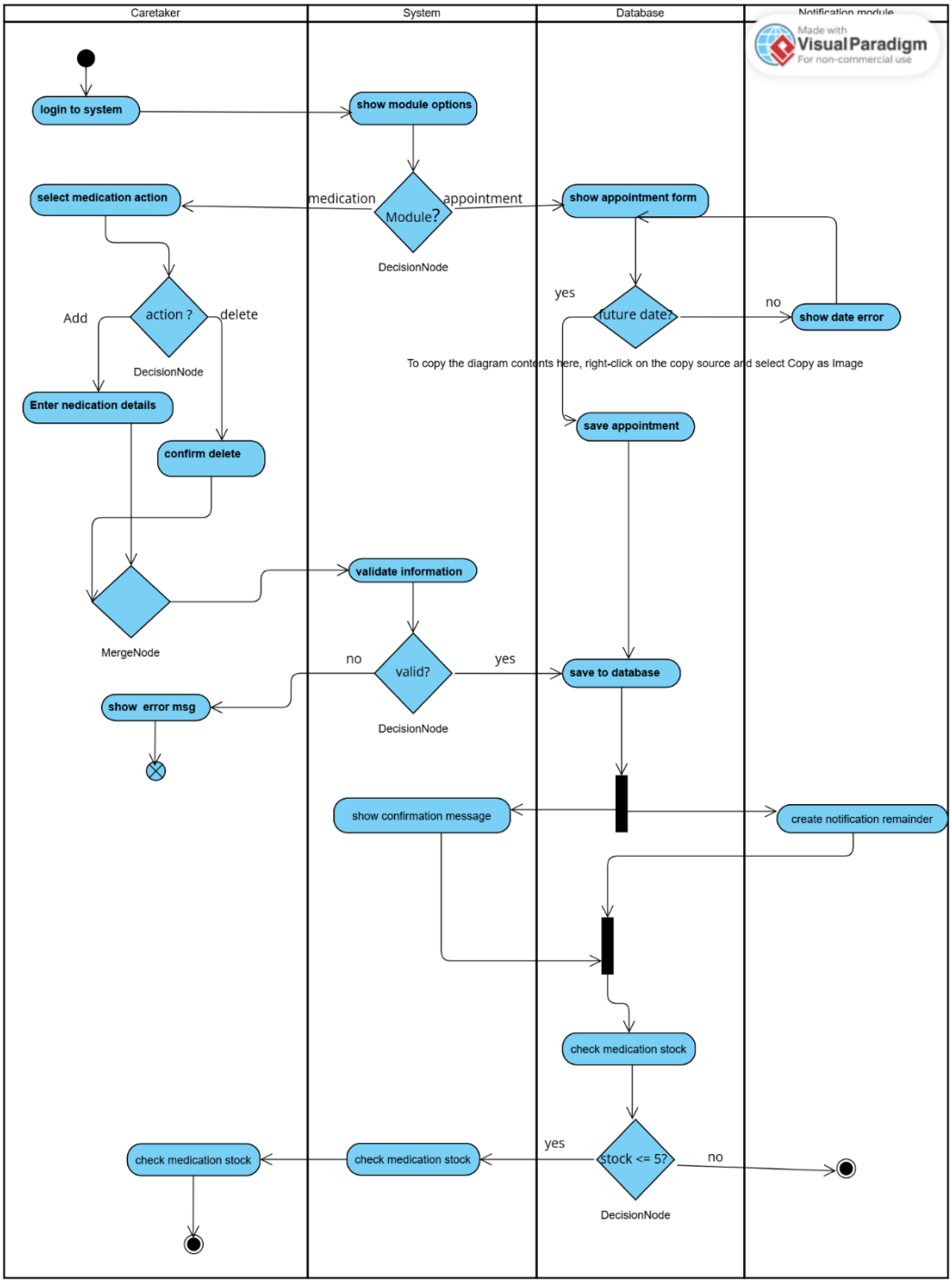
A new user selects the registration option and fills in personal details. The system validates the entered data and checks for duplicate accounts. If successful, a new account is created in the database. The user receives confirmation and can proceed to login.

2.3.3 Manage notification



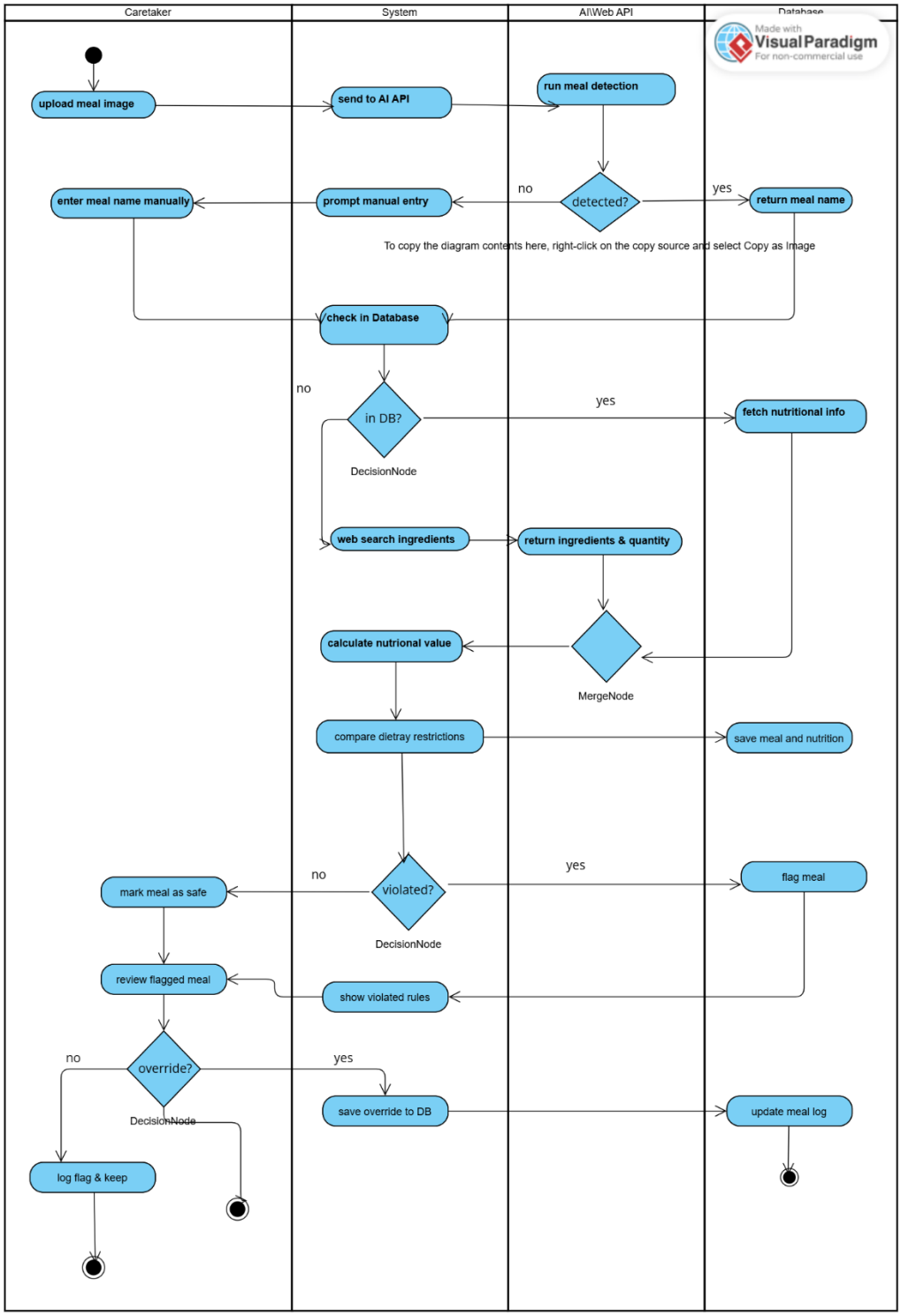
The system continuously checks medication and appointment timings. When a scheduled time arrives, a reminder notification is sent. The user can view, snooze, or dismiss notifications. All notification activities are logged for tracking.

.2.3.4 Medication and Appointment



The user adds medication schedules or books an appointment. The system stores medicine timing and doctor visit details. Notifications are generated for reminders. The user can update or cancel schedules anytime

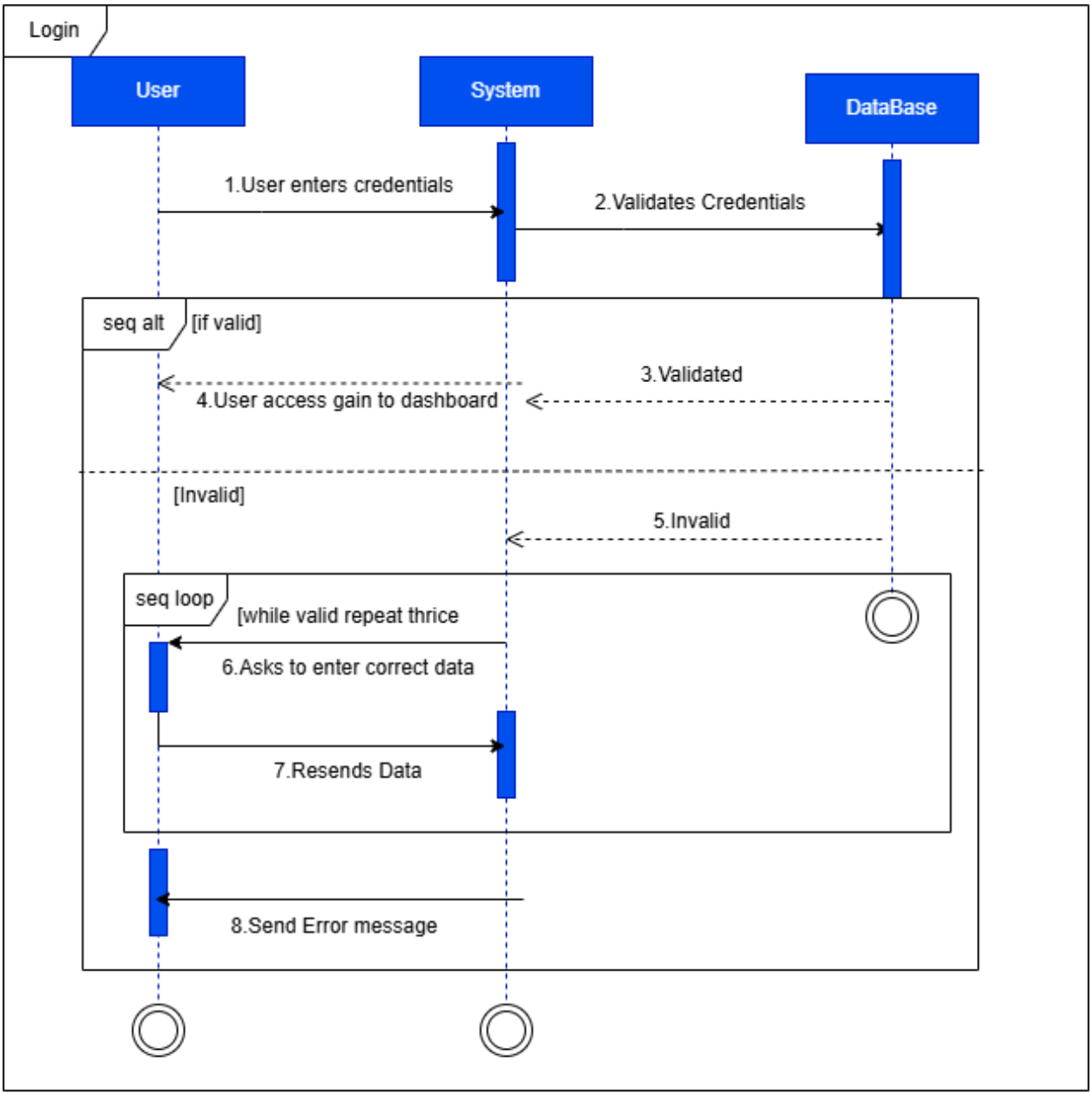
2.3.5 AI meal flagger



The user uploads meal details or selects food items. The AI module analyzes nutritional values and diabetes risk factors. If unhealthy food is detected, warning alerts are shown. The system also suggests healthier alternatives.

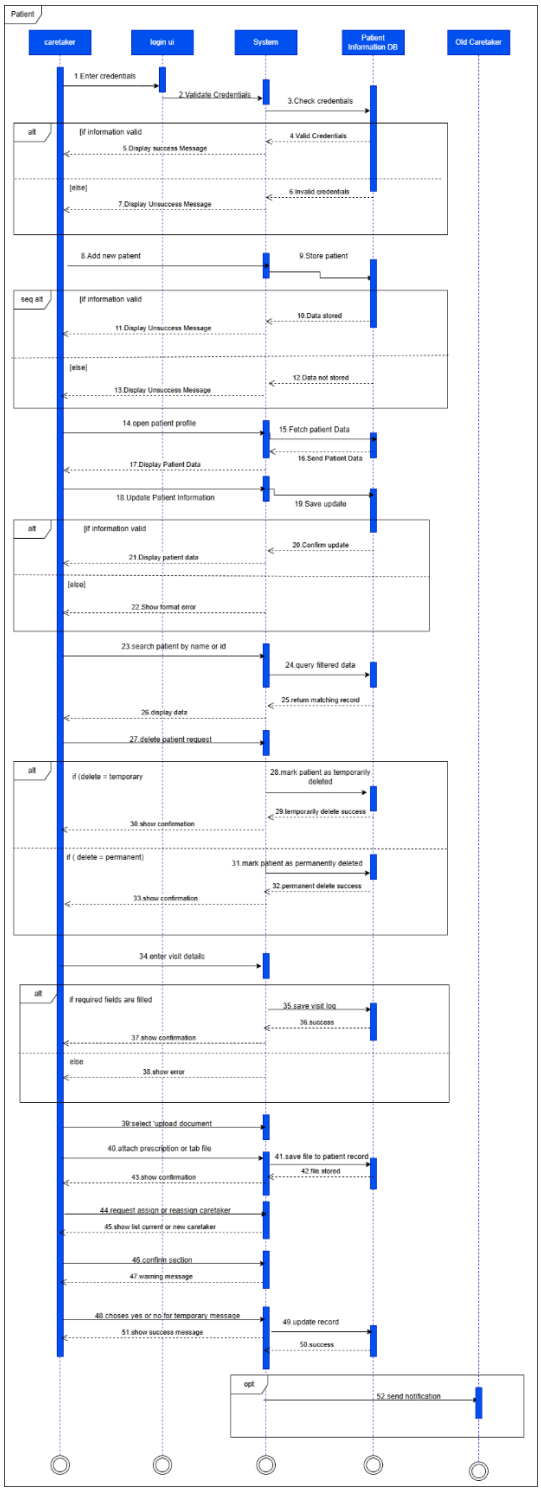
2.4 Sequence Diagrams

2.4.1 Login



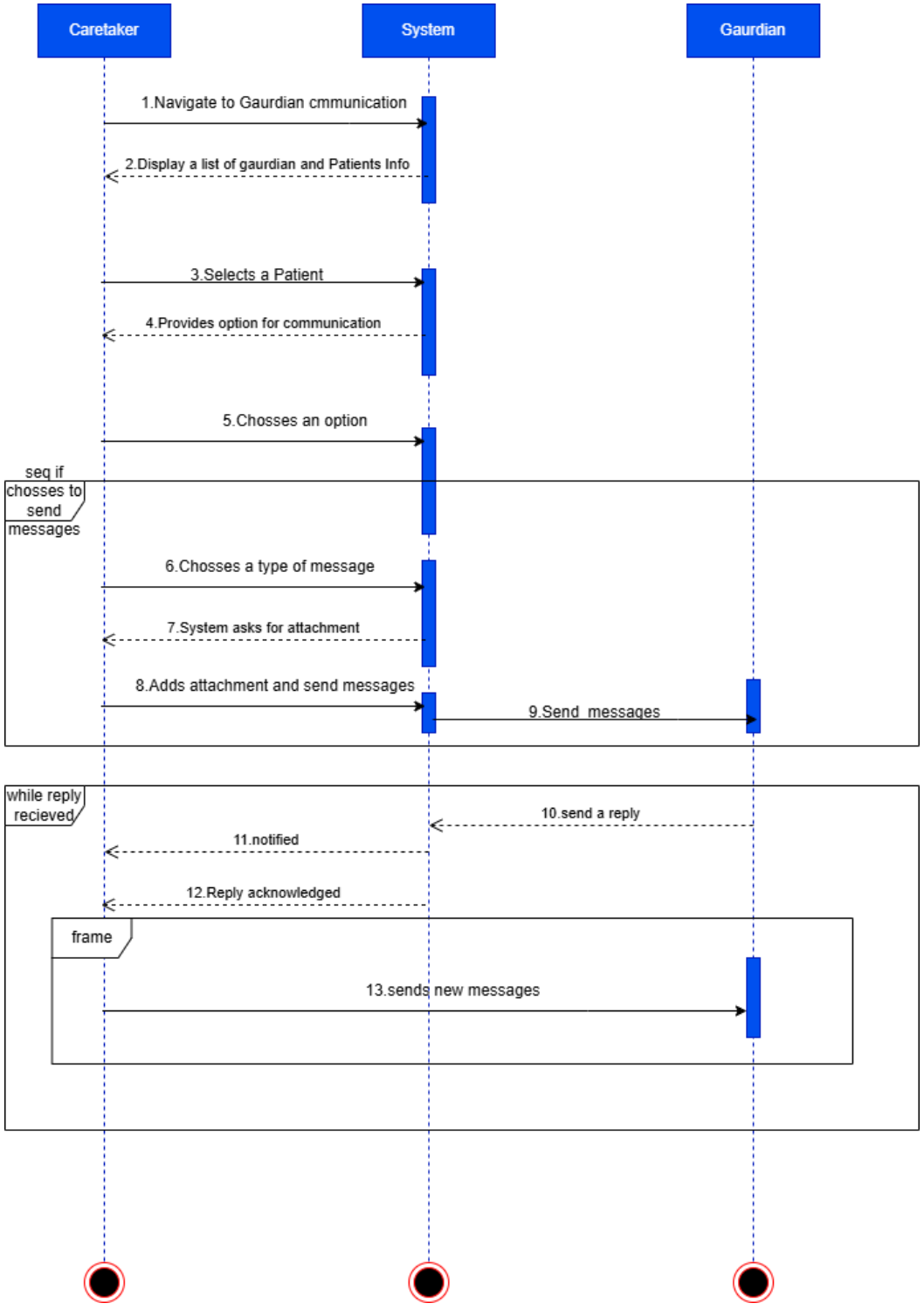
This diagram shows the dynamic flow of logging in. This feature is to be used by both the caretaker and guardian where their credentials will be verified before gaining access.

2.4.2 Manage Patient Records



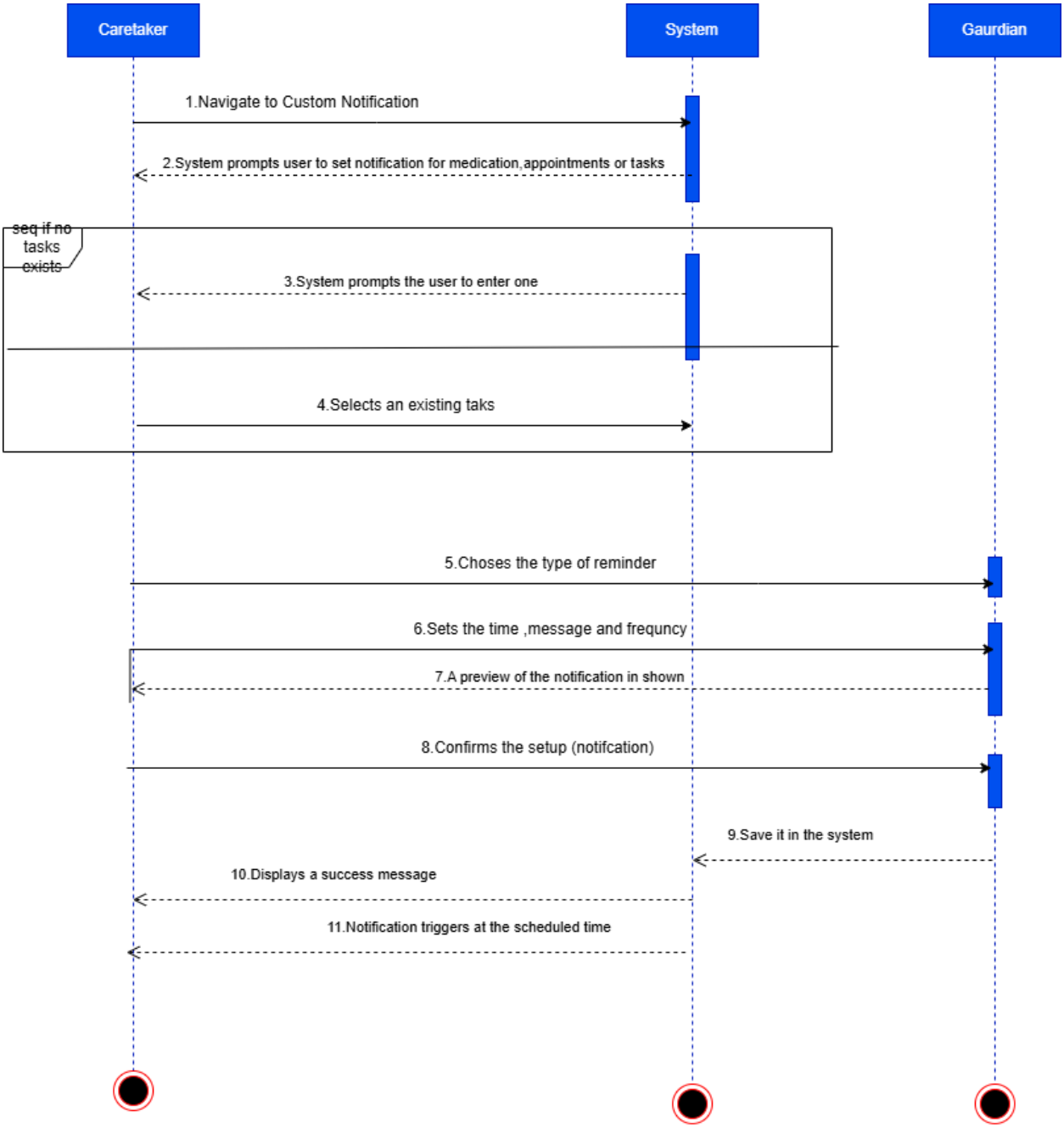
The caretaker logs into the system and accesses their dashboard to manage patient records. They can add a new patient by entering details, after which the system verifies the data and confirms if it's valid or highlights errors. The caretaker can update existing records, search for patients and choose to delete records temporarily or permanently, each with proper confirmation messages. After each visit, the caretaker logs visit details such as time and tasks completed. They can also upload prescriptions or lab reports, and assign or reassign caretakers, where the system confirms the changes and notifies the old caretaker.

2.4.3 Access Communication



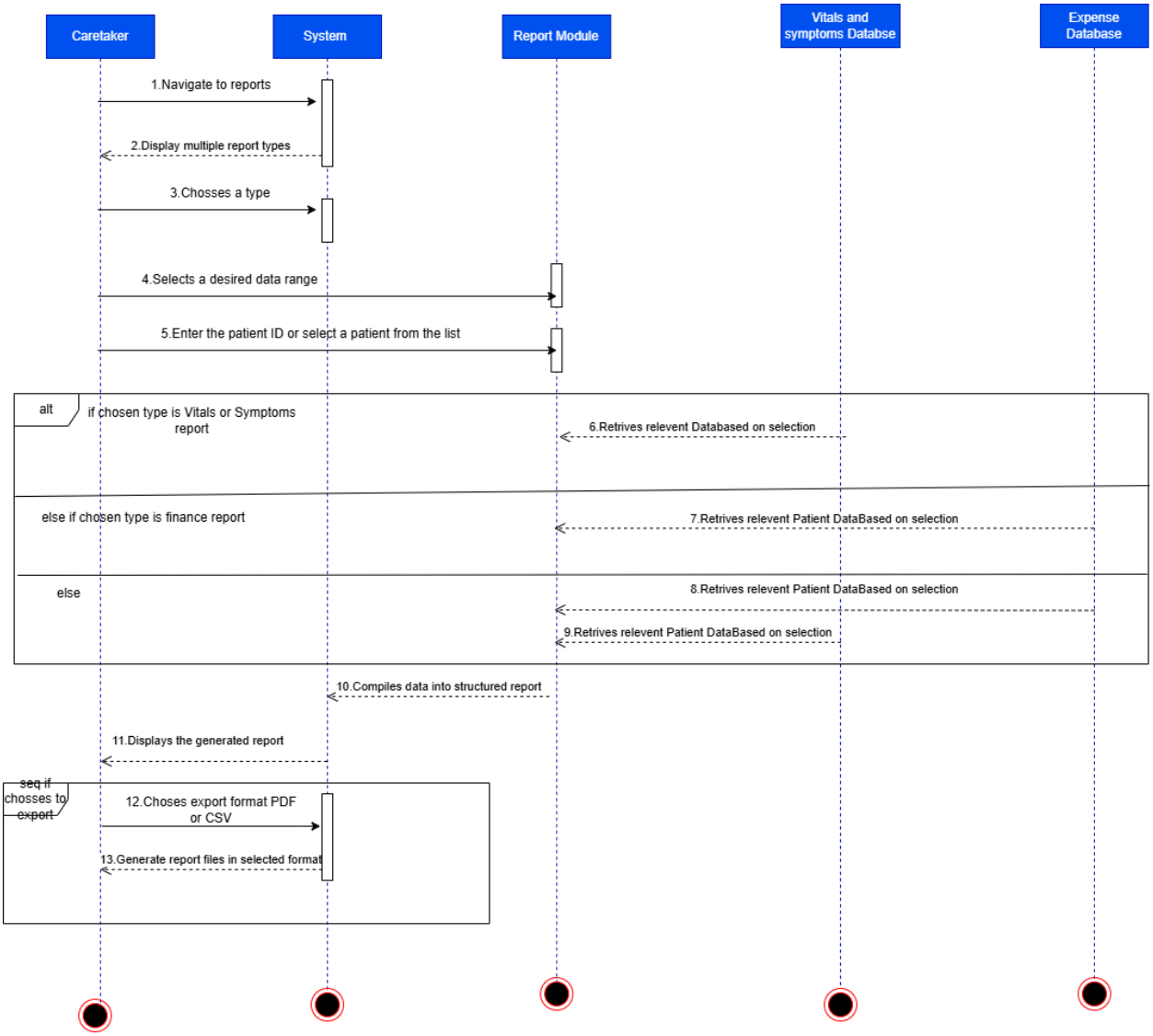
Caretaker logs in and navigates to the Guardian Communication section from the dashboard. The system displays a list of patients along with their associated guardians. The caretaker selects a patient and chooses one of the communication options. If the caretaker chooses to send a new message, they will select the message type (update, alert, query, etc.) so that the guardian will know the importance of that message. Along with the message, the caretaker can also attach relevant documents (such as nutritional or financial reports) and then send the message to the guardian via in-app messaging. The caretaker is notified of any responses and can continue the conversation if needed.

2.4.4 Manage notification



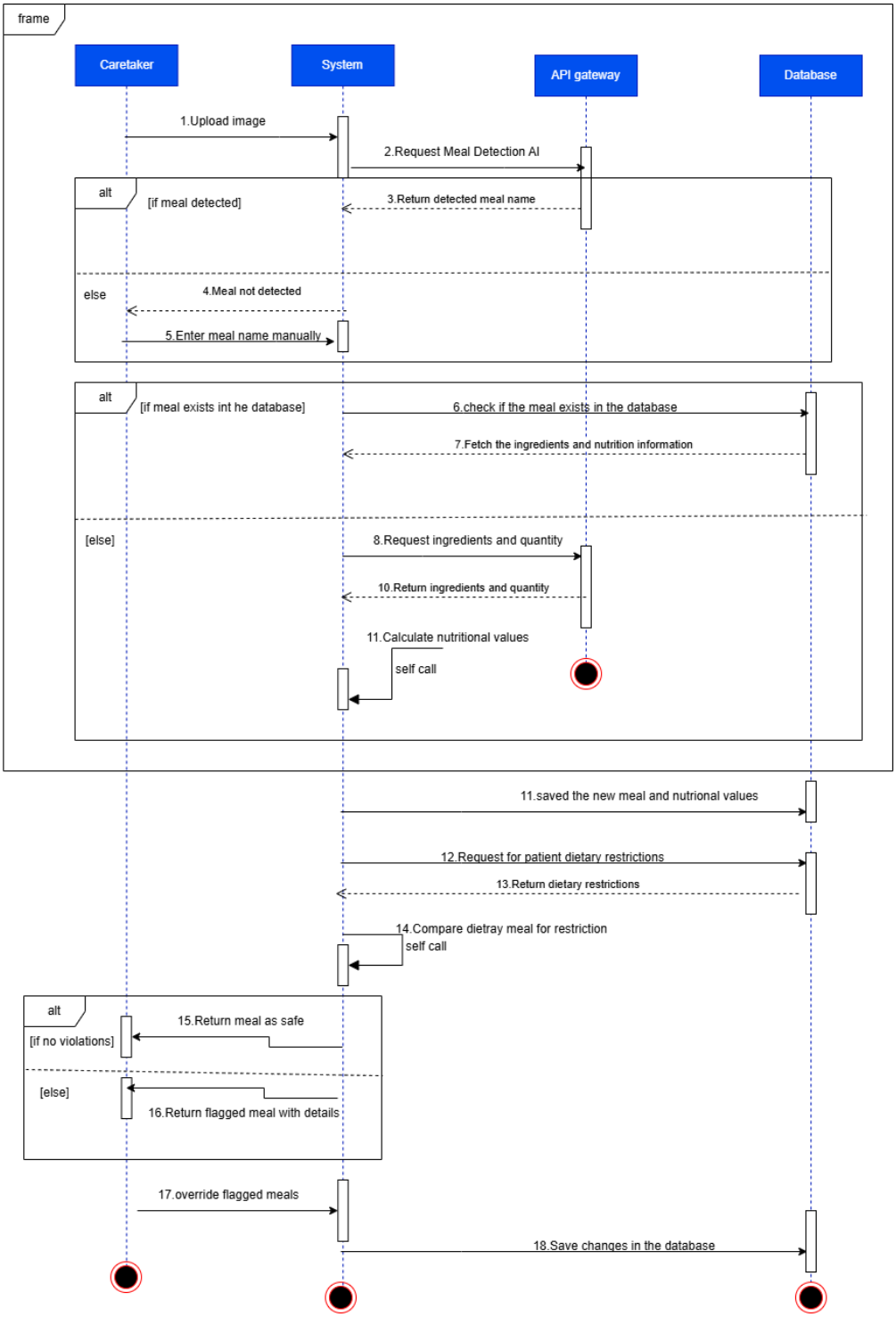
After signing in, the caretaker is given options to manage custom notifications for medications, patient appointments, or caretaker tasks. The caretaker selects an existing task, medication, or appointment to set a notification for. If none exist, the system prompts them to add one first. The caretaker chooses the type of reminder—daily, weekly, specific date/time, or custom interval—and sets the time, message, and frequency. Notifications can be delivered via push notification, SMS, or email. A preview of the notification is shown, and the caretaker reviews and confirms the setup before saving it in the system. A confirmation message appears once the notification is successfully added.

2.4.5 Access Patient Reports



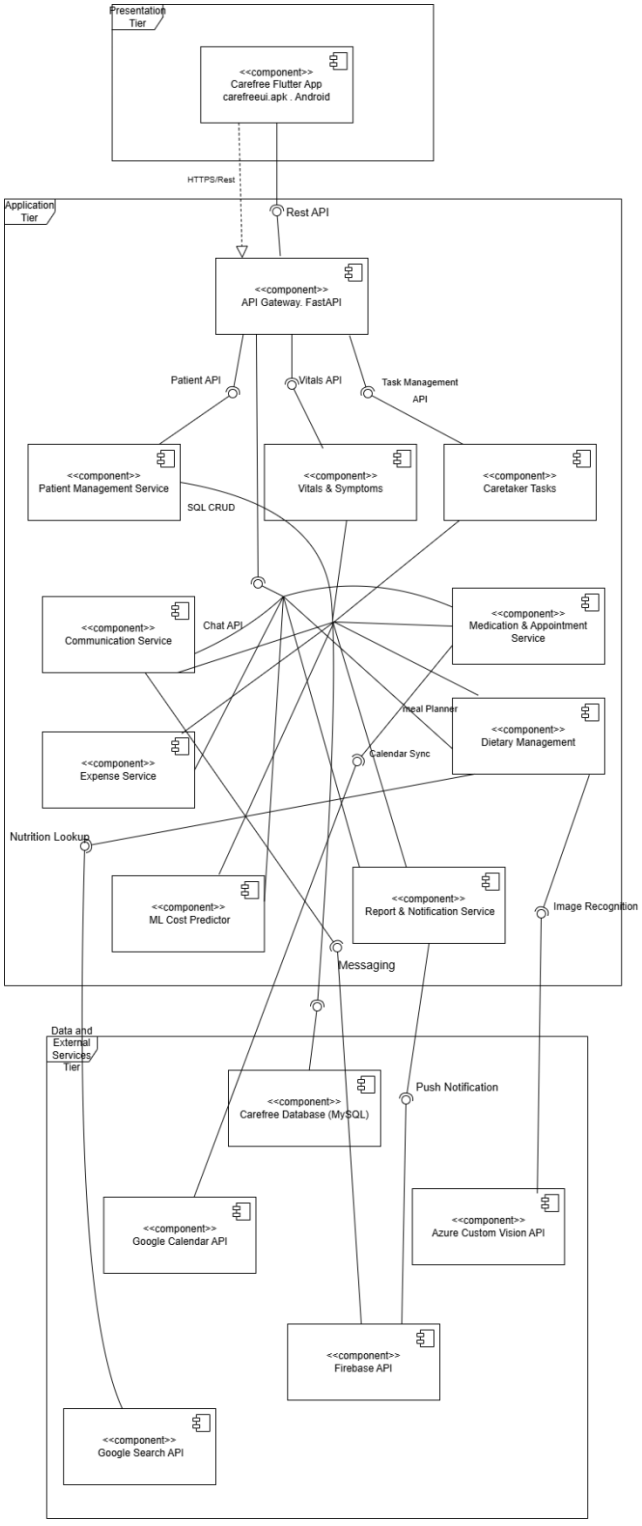
The caretaker or guardian logs into the system and is directed to the dashboard. From the dashboard, the user selects the "Reports" option to proceed. The system displays multiple report types to choose from. The user selects a desired date range for the report. The user enters the Patient ID or selects the patient from a list to specify which report to generate. The system fetches relevant data for the selected patient and time frame. Based on the selection, the system compiles the data into a structured report with summaries and alerts. The report is displayed to the user. The user can export the report as a PDF or CSV file for offline viewing or sharing

2.4.6 Upload Photo to AI Meal Flagger



The caretaker initiates the meal detection process. The system uses an AI-based meal detector to identify the meal and its ingredients via web APIs, then cross-checks this information against the patient's dietary restrictions to determine if the meal should be flagged.

2.5 Component Diagram



The UML Component Diagram of the Carefree Healthcare Management System shows the overall structure of the system and the interaction between its major components. It includes the Flutter mobile application, API Gateway, backend services, MySQL database, and external APIs such as Firebase and Google Calendar. Components communicate through provided and required interfaces using ports and assembly connectors. The diagram highlights dependencies between modules and demonstrates a modular, scalable, and maintainable system design.

2.6 Deployment Diagram

The deployment diagram maps all software components onto their physical or virtual hardware nodes across the three architectural tiers of the Carefree system. Nodes communicate via internet connections using REST APIs.

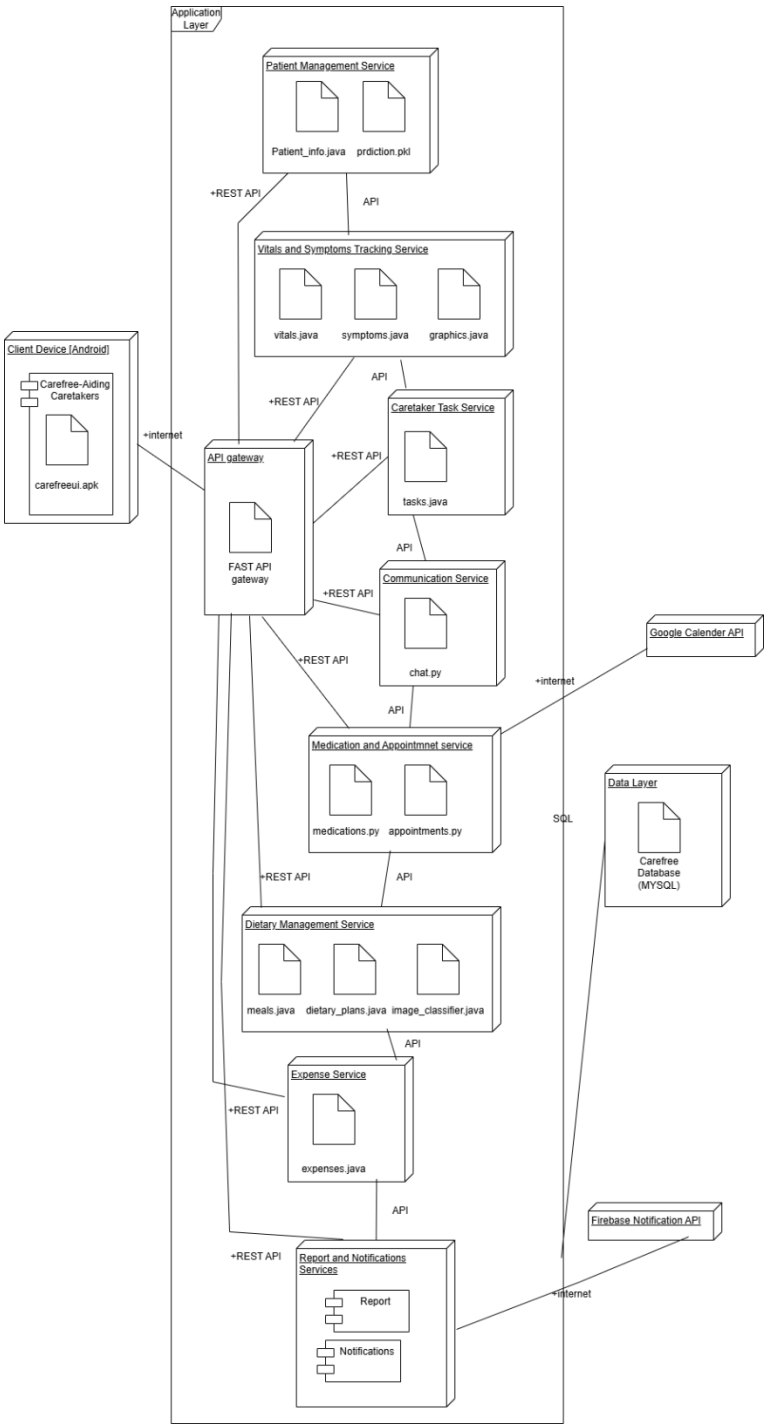


Figure 6 System Deployment Diagram — Carefree

2.6.1 Node Descriptions

Node	Layer	Components	Description
Client Device [Android]	Presentation	Carefree-Aiding Caretakers app, carefreeui.apk	Hardware Android device running the Flutter APK. Connects to the API Gateway via internet with secure encrypted communication.
API Gateway	Application	FAST API gateway	FastAPI-based secure gateway that handles all API requests from the Flutter frontend. Routes traffic to the correct backend microservice, enforces authorisation, and acts as the single entry point for all client requests.
Patient Management Service	Application	Patient_info.java, prdiction.pkl	Manages patient CRUD operations and exposes the ML cost prediction endpoint using the trained gradient boosting model (prediction.pkl).
Vitals and Symptoms Tracking Service	Application	vitals.java, symptoms.java, graphics.java	Records and retrieves patient daily vitals and symptoms. Also handles graphical trend generation for health data visualisation.
Caretaker Task Service	Application	tasks.java	Manages caretaker to-do tasks including creation, editing, deletion, and progress tracking.
Communication Service	Application	chat.py	Handles real-time in-app messaging between caretakers and guardians, including message-type classification and document attachment support.
Medication and Appointment Service	Application	medications.py, appointments.py	Manages medication schedules and patient appointments, including CRUD operations and notification triggers.
Dietary Management Service	Application	meals.java, dietary_plans.java, image_classifier.java	Manages dietary plans and meals. Integrates with the Azure Custom Vision API for AI-based image meal detection and dietary restriction checking.
Expense Service	Application	expenses.java	Handles all patient expense logging, editing, deletion, and financial report generation.
Report and Notifications Services	Application	Report module, Notifications module	Generates structured reports (vitals, symptoms, finances) and dispatches notifications via Firebase to users based on configured triggers.
Data Layer	Data	Carefree Database (MYSQL)	Centralised MySQL relational database accessed by all microservices via SQL queries. Stores all patient, caretaker,

			appointment, medication, expense, task, and notification data.
Google Calendar API	External	—	Cloud service allowing the Caretaker Task Service and Appointment Service to store and sync appointment and medication schedules.
Azure Custom Vision API	External	—	Cloud AI service enabling the Dietary Management Service to perform image-based meal detection for the AI Meal Flagger feature.
Firebase Notification API	External	—	Cloud messaging service enabling the Report & Notification Service to deliver push notifications and support real-time in-app communication.

CHAPTER 3: ARCHITECTURAL VIEW (4+1 MODEL)

The 4+1 architectural view model provides a comprehensive multi-perspective description of the Carefree system. Each view addresses different stakeholder concerns and together they form a complete architectural picture.

3.1 Logical View

The Logical View describes the system's object-oriented decomposition, focusing on the key abstractions and their relationships.

The system is organized around the Person base class, from which Patient, Guardian, and Caretaker are derived. The Patient entity is the central data object, aggregating Symptoms, Medication, Vitals, Appointment, DietaryPlan, and Expense objects. The Caretaker manages Tasks and triggers Notifications. The Vitals class uses inheritance to accommodate three sub-types: Cardiac, Respiratory, and Others.

Package / Layer	Key Classes	Responsibility
User Domain	Person, Patient, Guardian, Caretaker	Core user and patient data modeling
Health Tracking	Vitals, Cardiac, Respiratory, Others, Symptoms	Patient health measurements and symptom logging
Care Management	Medication, Appointment, Task, DietaryPlan, Meal	Day-to-day caregiving activities
Financial	Expense	Patient expense tracking and reporting
Communication	Notification	Alerts, reminders, and messaging

3.2 Process View

The Process View addresses concurrency, threading, and control flow across the system. Carefree operates on a three-tier SOA architecture with the following concurrent service threads:

- Patient Management Service — handles patient CRUD operations and ML prediction requests
- Vitals & Symptoms Tracking Service — continuously receives and analyzes patient data; generates alerts asynchronously
- Medication & Appointment Service — manages schedule entries and coordinates with the Notification service
- Dietary Management Service — processes AI meal detection requests via the Azure Custom Vision API asynchronously
- Communication Service — real-time messaging between Caretaker and Guardian via Firebase
- Report & Notification Service — asynchronously compiles report data and dispatches notifications

The API Gateway (FastAPI) routes all incoming requests from the Flutter client to the appropriate microservice, ensuring no single service is a bottleneck. All services communicate with the shared MySQL database via SQL queries, though each service owns a logical partition of the schema.

3.3 Development View

The Development View describes the system's module and package organization for developers.

Module	Technology	Files / Components
Frontend (Presentation)	Flutter (Dart)	carefreeui.apk; screens for Dashboard, Vitals, Meals, Tasks, etc.
API Gateway	FastAPI (Python)	FAST API gateway; routes requests to backend services
Patient Service	Spring Boot (Java)	patient_info.java, prediction.pkl
Vitals Service	Spring Boot + Python	vitals.java, symptoms.java, graphs.py
Caretaker Task Service	Spring Boot (Java)	tasks.java
Communication Service	Python	chat.py
Medication & Appointment	Python	medications.py, appointments.py
Dietary Management	Spring Boot + Python	meals.java, dietary_plans.java, image_classifier.pkl
Expenses Service	Spring Boot (Java)	expenses.java
Report & Notification	Mixed	Report module, Notifications module
Database Layer	MySQL	Carefree Database (single instance)

3.4 Physical View

The Physical View maps software components onto hardware infrastructure.

Node	Layer	Description
Client Device (Android)	Presentation	Android smartphone running the Carefree Flutter APK; connects via internet to the API Gateway
API Gateway Server	Application	FastAPI-based gateway server routing requests to backend microservices; handles authentication and load balancing
Application Server	Application	Hosts all Spring Boot and Python microservices; each service deployed as an independent process
Database Server	Data	MySQL server hosting the Carefree Database; accessed via SQL from all microservices

Google Calendar API	External	Cloud service for appointment scheduling and calendar sync
Azure Custom Vision API	External	Cloud AI service for image-based meal detection
Firebase Notification API	External	Cloud service for push notifications and real-time messaging

3.5 +1 Use Case View

The Use Case View ties together all four architectural views by showing how users interact with the system through its primary use cases. The key scenarios that drive the architecture are:

- Login — triggers the authentication flow across the API Gateway and Database
- Manage Patient Records — exercises the Patient Management Service and MySQL data layer
- Vitals & Symptom Tracking — demonstrates concurrent data recording, analysis, and alert generation
- AI Meal Flagger — illustrates integration of the Flutter client, API Gateway, Azure Custom Vision API, and Dietary Management Service
- ML Medical Cost Predictor — shows the full ML pipeline from data retrieval to model inference and result storage
- Guardian Communication — demonstrates real-time messaging via the Communication Service and Firebase

These scenarios are fully modeled in the Sequence Diagrams (Section 2.4) and State Chart Diagrams (referenced in the full SDD).

CHAPTER 4: REFERENCES

- IEEE Std 1016-2009: IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE, 2009.
- Unified Modeling Language (UML) Specification, Version 2.5.1. Object Management Group (OMG), 2017.
- Flutter Documentation. Google LLC. https://flutter.dev/docs
- Spring Boot Reference Documentation. VMware, Inc. https://spring.io/projects/spring-boot
- FastAPI Documentation. Sebastián Ramírez. https://fastapi.tiangolo.com
- Azure Custom Vision API Documentation. Microsoft Corporation. https://learn.microsoft.com/en-us/azure/cognitive-services/custom-vision-service/
- Firebase Cloud Messaging Documentation. Google LLC. https://firebase.google.com/docs/cloud-messaging
- Google Calendar API Documentation. Google LLC. https://developers.google.com/calendar/api
- MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/
- HIPAA Privacy Rule Summary. U.S. Department of Health & Human Services. https://www.hhs.gov/hipaa/